UPLOADING AND LOADING THE DATASET

```
import pandas as pd
import numpy as np
from google.colab import files


# Load into Pandas DataFrame
base_url = "https://raw.githubusercontent.com/Group-22-AI-ML/medical-diagnosis-

df = pd.read_csv(base_url)
```

INSPECTING AND UNDERSTANDING THE DATA

```
# Check columns and data types
print("Data Summary")
df.info()

# Identify target column distribution
print("\n--- Target Class Distribution ---")
print(df['class'].value_counts())
print("\n--- Target Class Percentage ---")
print(df['class'].value_counts(normalize=True) * 100)
```

```
Data Summary
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 520 entries, 0 to 519
Data columns (total 17 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Age                 520 non-null    int64
 1   Gender              520 non-null    object
 2   Polyuria            520 non-null    object
 3   Polydipsia          520 non-null    object
 4   sudden weight loss  520 non-null    object
 5   weakness            520 non-null    object
 6   Polyphagia          520 non-null    object
 7   Genital thrush      520 non-null    object
 8   visual blurring     520 non-null    object
 9   Itching             520 non-null    object
 10  Irritability        520 non-null    object
 11  delayed healing     520 non-null    object
 12  partial paresis     520 non-null    object
 13  muscle stiffness    520 non-null    object
 14  Alopecia            520 non-null    object
 15  Obesity             520 non-null    object
 16  class               520 non-null    object
dtypes: int64(1), object(16)
memory usage: 69.2+ KB
```

```
--- Target Class Distribution ---
class
Positive    320
Negative    200
Name: count, dtype: int64

--- Target Class Percentage ---
class
Positive    61.538462
Negative    38.461538
Name: proportion, dtype: float64
```

CHECKING DATA QUALITY

```
# 1. Check for missing values across all columns
missing_counts = df.isnull().sum()
print("Missing values per column:\n", missing_counts)

# 2. Check for duplicate rows
num_duplicates = df.duplicated().sum()
print(f"\nTotal duplicate patient records found: {num_duplicates}")

# 3. Check unique values per column for formatting inconsistencies
print("\n--- Unique values in each column ---")
for col in df.columns:
    print(f"{col}: {df[col].unique()}")

# 4. Outlier check on numerical variable 'Age'
print("\n--- Age Statistics ---")
print(df['Age'].describe())
```

```
Missing values per column:
 Age                  0
Gender                0
Polyuria              0
Polydipsia            0
sudden weight loss    0
weakness              0
Polyphagia            0
Genital thrush        0
visual blurring       0
Itching               0
Irritability          0
delayed healing       0
partial paresis       0
muscle stiffness      0
Alopecia              0
Obesity               0
class                 0
dtype: int64

Total duplicate patient records found: 269
```

```
--- Unique values in each column ---
Age: [40 58 41 45 60 55 57 66 67 70 44 38 35 61 54 43 62 39 48 32 42 52 5
 49 63 30 50 46 36 51 59 65 25 47 28 68 56 31 85 90 72 69 79 34 16 33 64
 27 29 26]
Gender: ['Male' 'Female']
Polyuria: ['No' 'Yes']
Polydipsia: ['Yes' 'No']
sudden weight loss: ['No' 'Yes']
weakness: ['Yes' 'No']
Polyphagia: ['No' 'Yes']
Genital thrush: ['No' 'Yes']
visual blurring: ['No' 'Yes']
Itching: ['Yes' 'No']
Irritability: ['No' 'Yes']
delayed healing: ['Yes' 'No']
partial paresis: ['No' 'Yes']
muscle stiffness: ['Yes' 'No']
Alopecia: ['Yes' 'No']
Obesity: ['Yes' 'No']
class: ['Positive' 'Negative']

--- Age Statistics ---
count    520.000000
mean      48.028846
std       12.151466
min       16.000000
25%       39.000000
50%       47.500000
75%       57.000000
max       90.000000
Name: Age, dtype: float64
```

REVIEWING THE FEATURES

```
# Standardize column headers: trim spaces, lowercase, replace spaces with under
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# Select the 9 target features + target column
selected_features = [
    'age',
    'gender',
    'polyuria',
    'polydipsia',
    'sudden_weight_loss',
    'weakness',
    'polyphagia',
    'genital_thrush',
    'visual_blurring',
    'obesity'
]
target_col = 'class'

# Filter dataset to selected features only
```

```
df_cleaned = df[selected_features + [target_col]].copy()
print("Selected columns:", df_cleaned.columns.tolist())
```

```
Selected columns: ['age', 'gender', 'polyuria', 'polydipsia', 'sudden_wei
```

CONVERTING CATEGORICAL FEATURES TO NUMBERS (0,1)

```
# Encode Gender: Male -> 1, Female -> 0
df_cleaned['gender'] = df_cleaned['gender'].map({'Male': 1, 'Female': 0})

# Encode Symptoms: Yes -> 1, No -> 0
symptom_columns = [
    'polyuria', 'polydipsia', 'sudden_weight_loss',
    'weakness', 'polyphagia', 'genital_thrush', 'visual_blurring', 'obesity'
]
for col in symptom_columns:
    df_cleaned[col] = df_cleaned[col].map({'Yes': 1, 'No': 0})

# Encode Target: Positive (Diabetic) -> 1, Negative (Non-diabetic) -> 0
df_cleaned['class'] = df_cleaned['class'].map({'Positive': 1, 'Negative': 0})

# Verify all columns are now numeric (int64)
df_cleaned.dtypes
```

	0
age	int64
gender	int64
polyuria	int64
polydipsia	int64
sudden_weight_loss	int64
weakness	int64
polyphagia	int64
genital_thrush	int64
visual_blurring	int64
obesity	int64
class	int64

dtype: object

```
from google.colab import files

# Save the full cleaned dataframe to CSV
df_cleaned.to_csv('cleaned_diabetes_data.csv', index=False)
```

```
# Trigger automatic download to your device
files.download('cleaned_diabetes_data.csv')
```